# cc-review Guide

Verified against the baseline captured on 2026-11-05

### Pinned sources

**cc-review** (ref v1.4.0)
https://github.com/rhuss/cc-review
commit `37efe889dec45f580b8f33307790701d71cab74a`

# Contents

**Overview**

- cc-review Guide

**Using cc-review**

- Using cc-review
- Install cc-review
- Review your changes
- Triage pull request comments
- Inbox and issue assessment

**How it works**

- How it works
- The review pipeline
- Reviewers
- Meta-review
- Gate and fix
- Measured results

**Extending cc-review**

- Extending cc-review
- Write a prompt reviewer
- Write a command reviewer
- Trust and configuration

**Reference**

- Reference
- CLI reference
- Configuration and finding schema
- Invoking cc-review across harnesses

# Overview

# cc-review Guide

cc-review runs a panel of focused code-review agents, together with external review tools, over a set of changes. It merges their findings, removes the duplicates, filters what is left through two meta-review critics, fixes the Critical and Important issues and re-reviews them, and hands you one report.

That report is the outcome. The problem it addresses is the noise of ordinary AI code review: a single reviewer tends to leave a long list of comments, many of them low-confidence or already handled in the code, and running several review tools by hand means reading and reconciling each one yourself. cc-review does that reconciliation for you and reports the issues worth acting on.

## What the parts are

**cc-review** is a plugin that dispatches specialized code-review agents at your changes, merges and de-duplicates their findings with those of external tools, and runs an autonomous fix loop on the Critical and Important issues before showing you the report.

### The harnesses

A harness is an AI coding agent that runs in your terminal, editor, or desktop and carries out coding tasks from plain-language instructions. cc-review installs into three of them as a plugin, and each harness reads its own manifest and ignores the others.

- Claude Code is Anthropic's coding agent for the terminal, IDE, and desktop.
- Codex is OpenAI's coding agent.
- OpenCode is an open-source coding agent for the terminal.

### The external review tools

cc-review can also call external, command-line review tools and fold their output into the same report. It runs each one as a "command reviewer", reads the findings it prints, and merges them with the agents' findings.

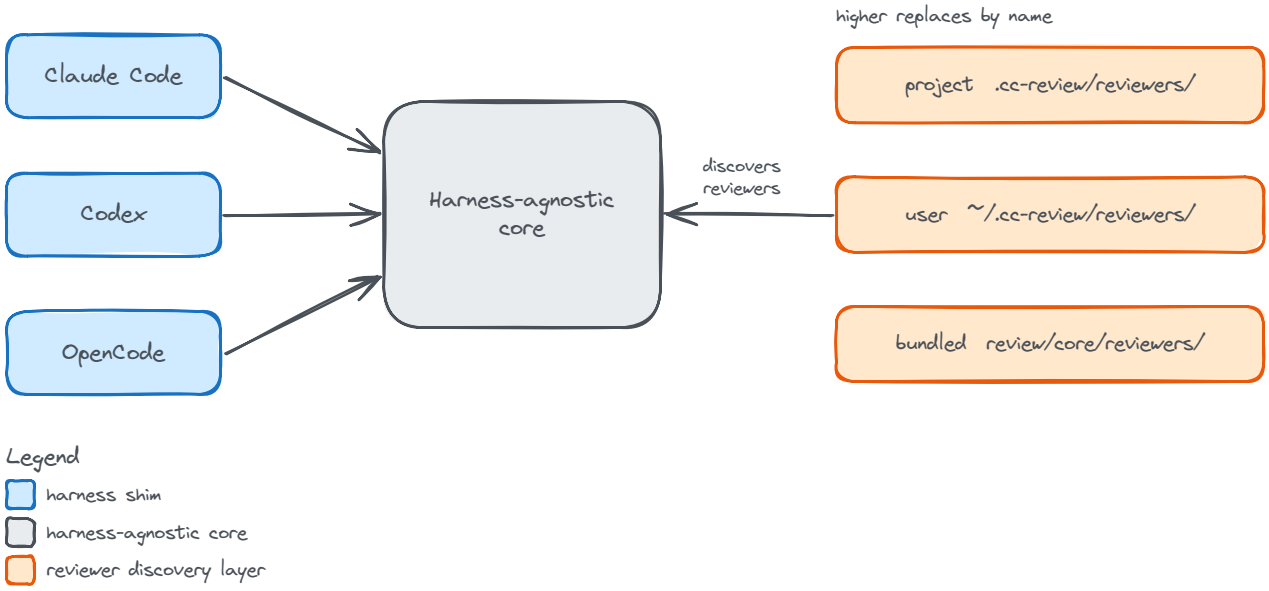
- CodeRabbit is an AI code review CLI, run by default.
- GitHub Copilot is GitHub's AI coding assistant; its command-line tool contributes review findings, and cc-review leaves it off by default.
- Codex CLI is OpenAI's Codex agent run as a reviewer, on by default.
- Greptile is a codebase-aware code review CLI, off by default.

> **Note**
>
> Codex shows up in both lists. As a harness it is the coding agent you install cc-review into; as Codex CLI it is one of the external reviewers that cc-review calls and merges with the rest.

# How the parts fit

cc-review keeps one harness-agnostic core and adds a thin manifest for each harness. The core holds the review logic; a manifest only tells its harness how to find and run that core, so a review behaves the same way whichever harness you start it from.

The review command carries no built-in list of reviewers. On every run, cc-review scans three layers and runs the reviewers it finds: your project's `.cc-review/reviewers/`, your personal `~/.cc-review/reviewers/`, and the reviewers bundled with the plugin. When a name appears in more than one layer, the higher layer wins, so you can add a reviewer or replace a bundled one without touching the core.




# Why run them together

Several focused reviewers and external tools from more than one vendor all read one change, and their results come back as a single filtered verdict rather than four separate comment streams. You see fewer, higher-confidence issues, and the Critical and Important ones are already fixed wherever cc-review could fix them.

# Status and limits

cc-review is at version 1.4.0. The external review tools are optional: cc-review runs each one only when its command-line tool is installed and enabled, and it ships with GitHub Copilot and Greptile turned off. Greptile also needs the repository connected to a Greptile organization, and it spends credits on every run (a base review costs one), which is why it stays off until you enable it. The Skeptic, one of the two meta-review critics, re-reads each finding against the code and rules it a false positive only when it can cite code that disproves it. When it cannot evaluate a finding, it keeps the finding, so it rarely drops one outright.

# Where to go next

- Developer: start with Using cc-review to install the plugin and run your first review.
- Plugin author: go to Extending cc-review to add your own reviewer or override a bundled one.
- To follow a review run from changes to report, read How it works.
- For commands, configuration, and the finding schema, see the Reference.

> **Note**
>
> This guide was verified against cc-review 37efe88 (version 1.4.0), captured 2026-11-05.

# Using cc-review

# Using cc-review

This section takes you from a fresh install to a steady review routine. You add cc-review to the coding agent you already run, whether that is Claude Code, Codex, or OpenCode, then put its commands to work on the code and pull requests in front of you. The payoff is a single merged report of the issues worth acting on, rather than a separate result from each reviewer and tool. You also get a clear view of which pull requests, issues, and discussions still need you next. Read the pages in order to get set up and build the routine, or jump straight to the command you need.

The pages, in reading order:

- Install cc-review adds the plugin to Claude Code, Codex, or OpenCode and lists the command-line tools to install first.
- Review your changes runs a review over your current branch or a specific pull request and explains how to read the report it returns.
- Triage pull request comments sorts a pull request's review comments into bot and human and drafts the replies for your approval.
- Inbox and issue assessment shows what needs your attention across a repository and helps you judge whether a new issue is worth the work.

# Install cc-review

cc-review ships as one plugin directory (`review/`) that carries a native manifest for each supported harness: Claude Code, Codex, and OpenCode. Each harness reads only its own manifest and ignores the others, so installing the plugin once registers the cc-review commands with the harness you use. Install it in the harness you already run.

> **Note**
>
> This guide's command examples use Claude Code syntax (`/cc-review:review`). On Codex use `$review`; on OpenCode use `/review`. See Invoking cc-review across harnesses.

## Prerequisites

cc-review needs a few command-line tools. Install these before you add the plugin:

- **`jq` 1.6 or newer.** Reviewer discovery and the inbox and issue commands use it.
- **mikefarah `yq` 4.18 or newer.** Reviewer discovery parses reviewer frontmatter with it. The Python `yq` and `yq` v3 use a different syntax and make every reviewer look malformed, so install the mikefarah build.
- **`awk`.** Reviewer discovery uses it alongside `jq` and `yq`.
- **`gh`, authenticated.** The inbox and issue commands call the GitHub CLI for the selected repository.
- **Python 3.9 or newer.** The inbox and issue commands run on it.

## Install on Claude Code

Claude Code installs cc-review through its plugin system. From a local clone of the repository, register it as a marketplace and install the plugin:

```
claude plugin marketplace add /path/to/cc-review
claude plugin install cc-review@cc-review-plugin-development
```

This registers the local marketplace and installs the plugin. The `/cc-review:review`, `/cc-review:triage`, `/cc-review:inbox`, and `/cc-review:issue` commands become available immediately.

> **Tip**
>
> From a local clone you can run `make install` instead. It registers the marketplace and installs the plugin in one step.

# Install on Codex

Codex uses the same plugin commands. From a local clone, register the marketplace and install the plugin:

```
codex plugin marketplace add /path/to/cc-review
codex plugin install cc-review@cc-review-plugin-development
```

The `review`, `triage`, `inbox`, and `issue` skills are then discovered from `review/skills/`.

# Install on OpenCode

OpenCode reads its commands from a directory inside your project. Run the install script from a local clone and point it at the project you want to review:

```
/path/to/cc-review/review/opencode/install.sh --project-dir .
```

This copies the cc-review commands to `.opencode/plugins/cc-review/` in your project.

Check: the plugin directory exists in your project after the script runs.

```
ls .opencode/plugins/cc-review/
```

# Install the spec-kit extension (optional)

> **Note**
>
> This step is only for projects that use cc-spex. Skip it otherwise.

Add cc-review as a spec-kit extension from a local clone:

```
specify extension add /path/to/cc-review/speckit
```

The spec-kit adapter then passes spec paths, constitution hints, and output locations during the ship pipeline.

# Duplicate command entries

If a harness lists each cc-review command twice, you most likely have more than one copy of the plugin installed. List the installed plugins (`claude plugin list` in Claude Code), uninstall every copy except the current one, then reinstall from your local clone.

> **Note**
>
> These steps were verified against cc-review 1.4.0.

# Review your changes

A review before you push catches the Critical and Important issues while the change is still on your machine, so you hand reviewers a cleaner pull request. The `/cc-review:review` command dispatches every enabled reviewer at your changes, merges the findings, and runs the fix loop over the issues worth acting on. This page covers how to start a review, how to narrow it, and how to read what it reports. For the mechanism behind each step, see How it works.

## Review the current branch

Start here when the change lives on your working branch and you have not opened a pull request yet.

1. Make sure your branch holds the changes you want reviewed, then run:

   ```
   /cc-review:review
   ```

   This reviews all changed files on the current branch against main, dispatches every enabled reviewer, merges their findings, and runs the fix loop.

2. Read the report when the run finishes. By default cc-review writes it to `./review-findings.md` in the repository root.

Check: the review wrote its report to the default path.

```
ls review-findings.md
```

## Review a pull request

Run a review against an open pull request when the change is already pushed, for your own PR or someone else's.

1. Review the diff of a specific PR by number:

   ```
   /cc-review:review --pr 42
   ```

2. When you run `--pr` from a branch other than the PR's own, cc-review checks the PR out into a separate worktree under `.cc-review/pr-worktrees/` and reviews it there, so your current working tree stays as it is. You do not need to switch branches first.

## Fixes and PR ownership

The autonomous fix loop applies its edits to the tree under review, but whether those edits reach the pull request depends on your push access: cc-review can push fixes only to a PR you can push to. It offers to apply fixes only when the PR is your own; on another author's PR it offers to post the review and nothing more. When it does fix your own PR, it commits the edits in the PR worktree and pushes them to the PR branch, never with force.

A push can still be refused. You may lack push permission on the branch, as with a fork PR, the branch may have moved since the review started, or the push may fail for another reason. In each case cc-review does not push the fixes and does not post a review, because the findings would then describe code that is not on the PR. It reports the problem in plain words, keeps the fix worktree so the commit is not lost, and tells you the worktree path. The review itself still runs on a PR you do not own, so you can read and post its findings; only the fixes cannot land there.

## Choose how deep to go

cc-review ships three profile presets you select with `--profile`. Each one sets which reviewers run, the confidence threshold, the severities that block the gate, and the number of fix rounds.

```
/cc-review:review --profile thorough
```

- **`quick`** runs only the correctness and security reviewers, with no command (external tool) reviewers and a single fix round. Only Critical findings block its gate.
- **`thorough`** runs all bundled reviewers except Gator with stricter thresholds and up to five fix rounds. Critical, Important, and Minor findings all block its gate.
- **`ci`** runs all bundled reviewers except Gator and Greptile (Greptile spends credits on every run), produces verbose output, asks no interactive questions, and does not post to the PR. Critical and Important findings block its gate.

## Narrow or skip work

These flags trim a run when you want a report only, want to leave the external tools out, or want to target a subset of reviewers.

- `--no-fix` skips the autonomous fix loop, so cc-review reports the findings without changing your code.
- `--no-external` disables all command (external tool) reviewers and runs only the bundled agents.
- `--only <reviewer,...>` dispatches only the reviewers you name, for example `--only correctness,security`. Any discovered reviewer name works here.
- `--spec <path>` cross-checks the implementation against a specification file; the goal alignment reviewer verifies that each functional requirement (FR-NNN) is implemented.

You can combine them. This produces a report only, from the bundled reviewers, with no fixes applied:

```
/cc-review:review --no-fix --no-external
```

## Read the report and the gate

cc-review presents the report for a person to read, not as a log. It opens with a verdict: one short paragraph on whether the change is ready to merge, how many issues you should address first, and the one or two themes behind them. It also names any reviewer that could not run, for example a tool over its usage limit. After the verdict, every Critical and Important finding gets its own section that explains where the problem is, what the code does today, and what goes wrong.

The gate is the pass-or-fix decision at the end. cc-review counts the findings whose severity blocks the gate (Critical and Important by default). When that count is zero the gate passes; when it is above zero the gate fails unless the fix loop clears those findings within the maximum rounds. Lower-severity notes do not affect the gate.

> **Note**
>
> The severities that block the gate come from the active profile. `quick` blocks on Critical alone, while `thorough` also blocks on Minor, so the same change can pass one profile and fail another.

## Review several PRs, and follow up

Run a bulk review to cover more than one pull request in a single pass:

```
/cc-review:review --bulk
```

This reviews the open PRs that request your review, plus the PRs you already reviewed with cc-review that have new commits since. Each PR is analyzed in its own worktree, and at the end you get one table to decide per PR: post the review, apply the suggested fixes, post a short no-findings note, or skip.

When you review the same PR again, cc-review runs a follow-up: it reviews only the delta since your latest completed cc-review run, including the code you changed to fix earlier findings, rather than re-reporting the whole diff.

> **Note**
>
> This page covers the commands you run. For how the reviewers, the two critics, and the fix loop produce the report, read How it works. For the full flag list, see the Reference.

> **Note**
>
> These commands were verified against cc-review 1.4.0.

# Triage pull request comments

An open pull request collects review comments from bots (CodeRabbit, Copilot, Devin, Greptile) and from human reviewers, and each one needs a reply or a fix. The `/cc-review:triage` command fetches the PR's review threads, classifies each as bot or human, and then handles them by kind: it works through the bot comments on its own and presents the human comments to you for approval. This page covers how to start a triage pass, how bot and human comments are handled, and why triage drafts replies instead of pushing a fix on your behalf. For the mechanism behind each step, see How it works.

## Triage the comments on a PR

Start here when a pull request has gathered review comments you want to work through. Triage needs an authenticated `gh` CLI. If `gh` is not authenticated, the command stops and asks you to run `gh auth login` first.

1. From the PR's branch, run:

   ```
   /cc-review:triage
   ```

   With no arguments, triage uses the open pull request for your current branch. When your branch has no PR but you have exactly one other open PR in the repository, triage uses that one and tells you which.

2. To target a specific pull request by number, pass `--pr`:

   ```
   /cc-review:triage --pr 42
   ```

   Triage includes draft PRs, so you can work through comments before the PR is marked ready for review.

Check: triage recorded its state for the PR.

```
ls .cc-review/.triage-state.json
```

## How bot and human comments are handled

Triage splits the review threads into bot threads and human threads and treats them differently.

Before it processes anything, triage logs every bot author it found and how many threads each one has, so no bot's comments slip through unseen. It matches each bot against a known profile (CodeRabbit, Copilot, Devin, and Greptile are recognized) and falls back to conservative defaults for any bot it does not know, but it still assesses every thread on its own.

For each bot thread, triage reads the comment and the code it points at, then reaches one of three verdicts:

- **Valid**: the comment identifies a real issue or improves the code, so triage applies the fix.

- **Invalid**: the code at the cited lines is already correct, so triage skips the fix and prepares a rejection reply that cites the lines proving the bot wrong.
- **Deferred**: the suggestion has merit but is out of scope for this PR, so triage records it and prepares a deferral reply.

Human threads are not handled automatically. Triage presents each one to you with the reviewer's comment, the code in question, and its own assessment (agree, disagree, or partial), together with a drafted reply. You then approve the reply as written, edit it before it posts, or skip the thread to leave it open for the next pass.

## Triage drafts replies, it does not push a reviewer fix

Triage works on comments on your own PR and on reviewer feedback you own on another author's PR. These two roles behave differently, and the difference matters when the PR is not yours.

> **Important**
>
> When you triage reviewer feedback you own, triage verifies the current code and the exact feedback thread and drafts a response for your approval. In that role it never commits or pushes a code fix. Author triage, on your own PR, keeps its fix workflow and applies valid bot fixes as described above.

For how the review command's fix loop handles the same question of which PRs it can push to, see Fixes and PR ownership.

## Add spec context and control coverage fixes

A few flags adjust what a triage pass does:

- `--spec <path>` gives triage a specification file so it can judge whether a suggestion aligns with the requirements.
- `--no-coverage-fix` skips the automatic coverage remediation.
- `--idea-inbox <path>` captures out-of-scope ideas in a file instead of acting on them.

When Codecov data is available for the PR, triage cross-references it with the bot findings and reports, per file, whether a coverage gap overlaps with an issue a bot already flagged. That overlap strengthens the case for accepting fixes that add error handling or branch coverage.

To triage several of your PRs in one pass, run `/cc-review:triage --bulk`. On your own PRs it keeps the prepared-fix workflow; for reviewer feedback it prepares exact-thread replies without changing any branch, and leaves answers that still need your judgment pending.

> **Note**
>
> This page covers the commands you run. For how triage verifies threads and classifies bots, read How it works. For the full flag list, see the Reference.

> **Note**
>
> These commands were verified against cc-review 1.4.0.

# Inbox and issue assessment

A repository collects pull requests, issues, and Discussions from many people, and not every one of them is yours to act on. Two commands help you decide where your attention goes before you start reviewing or writing code. `/cc-review:inbox` surveys one repository and groups the open items by who needs to act next. `/cc-review:issue` takes a single issue and separates its concrete need, its fit with the project, and whether the work is justified, then drafts a reply you can send. Both commands are read-only. Neither posts a comment, edits code, or changes an issue, so you can run either one to understand a situation without touching it.

Both commands need Python 3.9 or newer, `jq`, and an authenticated `gh` for the repository you work with.

Check: the tools both commands rely on are available.

```
gh --version && jq --version && python3 --version
```

## See what needs your attention

One repository gathers pull requests, issues, and Discussions from everyone who works on it, and the inbox explains which of them need you. It covers your own items, your assignments, review requests (including requests to your teams), mentions in pull requests and issues, and conversations you commented on or reviewed. The report groups every item by the action it is waiting on, so you read it top to bottom instead of opening each thread to find out.

1. From a checkout of the repository, run the command with no arguments:

   ```
   /cc-review:inbox
   ```

2. Read the report. It sorts every item into **Needs your attention**, **Waiting on others**, and **No current action**. Each entry says why it appears, links to the activity that supports it, and suggests a next step. A pull request can suggest both a review and a triage pass, because one does not clear the other.

The inbox keeps relevance apart from responsibility. It does not decide that you owe a reply because you authored an item, took part in it once, or someone posted after you. For each item it works out why the item appears, the current state, any outstanding request and who that request falls to, and the step you would take next. When the evidence does not settle who acts next, the item is marked uncertain and asks you to check the context, which is not a claim that you owe an answer.

## Narrow the inbox

By default the inbox reports all three item types: open pull requests and issues (including drafts) with no date cutoff, and Discussions updated in the last 90 days. Four flags narrow that scope.

- `--repo owner/name` picks a repository explicitly, even when you are not inside a checkout of it. Without the flag the inbox follows the repository that `gh` is pointed at.

- `--type pr,issue,discussion` selects one or more item types. Repeat the flag to combine selections; the default is all three.
- `--limit N` keeps the most recently updated items per type, from 1 to 100. The default is 10 per type.
- `--since YYYY-MM-DD` filters every selected type by its last update, and replaces the 90-day Discussion window.

For example, this reports only pull requests, up to 20 of them, updated since the start of October:

```
/cc-review:inbox --type pr --limit 20 --since 2026-10-01
```

## Assess a single issue

Before you commit time to an issue, you want to know whether it describes a real need, whether that need fits the project, and whether building it is justified. `/cc-review:issue` reads the issue conversation and any matching project guidance, makes those three judgments separately, and gives you both the reasoning and a reply you can post.

Assessment works on GitHub.com issues only, the target must be an issue (pull requests are rejected), and you name it explicitly.

1. Run the command with the issue number:

   ```
   /cc-review:issue 42
   ```

   You can also pass a full issue URL, or add `--repo owner/name` to assess an issue in a repository you are not checked out in.

2. Read the two parts of the result. The first is the maintainer reasoning, ending in one of four recommendations: **Pursue**, **Need more evidence**, **Clarify project direction**, or **Do not pursue**. The second is a separate friendly reply drafted for you to send. When the issue does not yet establish a concrete use case, the draft asks the author for one real example rather than proposing a solution.

## Both commands only read

Neither command changes anything on GitHub or in your checkout.

> **Important**
>
> The inbox runs no review, triage, checkout, assignment, merge, reply, or thread-resolution action; every suggested action is text for you to act on. Assessment never implements, posts, assigns, labels, or closes an issue, and it never launches a review. Posting a drafted reply is a separate step you take yourself.

> **Note**
>
> This page covers the commands you run. For how the inbox weighs responsibility and how assessment judges need and fit, read How it works. For the full flag list, see the Reference.

> **Note**
>
> These commands were verified against cc-review 1.4.0.

# How it works

# How it works

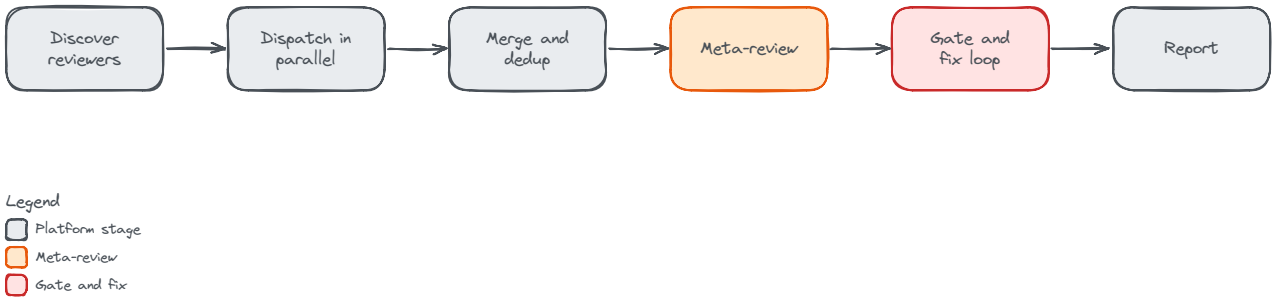
Every review run has the same shape. A panel of focused reviewers each read your changes in their own fresh context, two critics filter the findings they return, and a fix loop resolves the blocking ones and re-reviews until the gate is clear. What reaches you at the end is one report of fewer, higher-confidence issues, with the Critical and Important ones already fixed wherever cc-review could fix them, instead of a separate comment stream from every reviewer and tool. The pages below take that run apart one layer at a time.

The pages, in reading order:

- The review pipeline walks the stages a single run moves through, from discovering the reviewers to writing the final report.
- Reviewers describes the panel itself: the six built-in lanes, the external tools, the opt-in Gator persona, and the rules every reviewer follows.
- Meta-review covers the two critics, the Skeptic and the Calibrator, that judge the merged findings before anything is acted on.
- Gate and fix explains the pass-or-fix gate and the autonomous loop that clears the blocking findings so the gate can pass.
- Measured results reports how cc-review has performed in practice, as dated measurements from recorded runs on public and local code.

# The review pipeline

One review run carries a set of changes through six stages: discover, dispatch, merge, meta-review, gate and fix, and report. Each stage has a single job and hands its output to the next. The diagram below shows the order, with the fix loop as the one arrow that goes back.




## Discover

cc-review carries no built-in list of reviewers. Every run begins by discovering them from three layers: your project's `.cc-review/reviewers/`, your personal `~/.cc-review/reviewers/`, and the reviewers bundled with the plugin. Everything the run dispatches comes from this discovery output.

Each discovered reviewer is one of two kinds, `prompt` or `command`, and arrives with a status. Enablement then settles which reviewers will run. A reviewer is on or off according to its `reviewers.<name>` key in the merged configuration, or its own default when the configuration says nothing about it. Command-line flags override the configuration: `--only` narrows the run to a named set, `--no-<name>` drops one, and `--external` turns the external tools on or off as a group.

For the command reviewers that are still enabled, cc-review runs a short detection probe, bounded to 15 seconds by default, to confirm the tool is installed and authenticated. A reviewer that is off, cannot run, or fails its probe is marked accordingly and costs the run nothing further. What survives discovery is the ready set.

## Dispatch

The ready set splits by kind. The two kinds run in different ways, but at the same time.

A **prompt reviewer** is a review agent written as a prompt. cc-review dispatches each one as a separate subagent with a fresh context, so no reviewer sees another reviewer's findings or the orchestrator's state. The subagents run in parallel, read the changed files themselves, and return their structured findings through the `ReportFindings` tool, with their final text as a short summary. Keeping each reviewer in its own context also keeps the orchestrator's context small: it holds the findings, not the full contents of every file each reviewer read.

A **command reviewer** is an external command-line review tool. cc-review runs each one as a background process inside the tree under review, bounded by its own timeout, and reads the output it prints. Starting them all in the background and then waiting means the run costs the time of the slowest

tool rather than the sum of every timeout. cc-review parses each tool's output into the same finding shape the prompt reviewers use.

A reviewer that errors, times out, or returns nothing cc-review can parse is marked as failed. It never blocks the run; the remaining reviewers still report.

## Merge and de-duplicate

cc-review collects the findings from every reviewer that completed, prompt and command alike, and normalizes them into one shape. Each reviewer's own severity labels are translated into cc-review's severities once, at this point.

The merged findings are sorted by file and line, then de-duplicated. When two findings sit on the same file, cover overlapping lines, and share a category, cc-review keeps the fuller description, records the other reviewer under "also reported by", and takes the higher severity and confidence. Findings from a reviewer marked as isolated stay out of cross-category de-duplication, so a specialized lane keeps its own count. Each surviving finding gets a sequential id.

## Meta-review

Two critics read the merged findings and judge them before anything is acted on. This stage runs only when it is enabled and when the finding count reaches the configured threshold, five by default; a smaller run goes straight to the gate.

The **Skeptic** decides whether each finding is real, marking it confirmed, weak, or a false positive. The **Calibrator** sets the right severity, flags findings that contradict each other, and scores each reviewer. cc-review dispatches both as subagents in parallel, removes the false positives, applies the verdicts and calibrated severities, and annotates the contradictions. When the Skeptic and the Calibrator disagree, the more conservative outcome wins. If either critic fails, cc-review skips the meta-review and lets every finding through unchanged rather than dropping the run.

The meta-review page covers the two critics in detail.

## Gate and fix

The gate is a single decision: how many findings are Critical or Important. Zero is a pass. A count above zero fails the gate, unless the fix loop clears it. Notable findings never count toward the gate.

The fix loop runs when you ask for fixes. In each round, cc-review applies the suggested fixes, runs the test suite when it finds one, and re-dispatches the same reviewers on only the files it changed, then checks the gate again. A failing test becomes a Critical finding of its own. The loop runs up to a configured number of rounds, three by default, or stops early once the gate passes.

The gate and fix page covers the loop and its options.

## Report

Whatever the gate decides, the run ends by writing one findings report. It is written for someone who did not watch the run. The report opens with a summary of how many findings were found, fixed, and

left per severity, then lists each finding: where it is, what is wrong, why it matters, and how it was resolved. When the meta-review ran, the report also carries its per-reviewer scores and the findings it removed.

# Reviewers

A reviewer is one voice in the panel. cc-review ships three kinds: six built-in prompt reviewers, each a review agent with a single job; a set of external command-line tools it can call and fold into the same report; and one opt-in principal-engineer persona. The pipeline page covers how a run discovers and dispatches them. This page is about what each one is for and the rules they all follow.

## The six lanes

The bundled prompt reviewers divide the work into six lanes. Each lane owns one kind of problem, and nothing else.

- **Correctness** finds bugs: mutation safety, shared references, logic errors, resource cleanup, error-path correctness, off-by-one errors, null handling, and type confusion.
- **Architecture** judges structure and idioms: dead code, unnecessary complexity, duplication, misleading naming, comment accuracy, abstraction level, and convention adherence.
- **Security** looks for vulnerabilities: input validation, injection, secrets handling, authentication and authorization, path traversal, deserialization safety, and rate limiting.
- **Production Readiness** catches what fails in operation: resource leaks, unbounded growth, concurrency issues, error amplification, graceful shutdown, and observability gaps.
- **Test Quality** weighs the tests themselves: coverage gaps, weak assertions, tests that pass for the wrong reason, empty stubs, and missing edge cases.
- **Goal Alignment** checks that the change fits the project: the need and fit of the pull request description and its linked issues, whether the code delivers its stated goals, and whether it smuggles in undeclared changes. This lane needs a pull request to review, so it runs only when one is supplied.

### Stay in your lane

Each lane reviewer is told what it is responsible for, and then told what it is not. The Correctness reviewer is not responsible for style, test coverage, security, or architecture; the Security reviewer is not responsible for general correctness or production readiness; and so on around the panel. Every one of the six ends the same way: those concerns belong to other agents, so stay in your lane.

The role gate is what keeps the panel from collapsing into one blurry opinion repeated six times. Because each reviewer runs in its own fresh context and sees only its own slice of the problem, two reviewers rarely file the same issue, and the merge step has clean, single-category findings to work with rather than overlapping restatements.

## The shared preamble

Every prompt reviewer, the six lanes and the opt-in persona alike, is prepended with a common preamble. It sets the rules of evidence that make a finding trustworthy.

- **Judge what was built, not what was intended.** A reviewer distrusts the implementer's report, comments, and commit messages, and verifies everything by reading the actual code.

Comments may be wrong, names may mislead, and a test name may not match what the test checks.
- **Review the code and the spec, nothing else.** A reviewer does not read the git log, commit messages, brainstorm documents, or the plan and task files. Those reveal what the author meant to do and bias the review toward confirming it. The code and the spec are the only trustworthy record of what was built.
- **No praise.** A reviewer starts directly with its findings, not with an affirmation of the work.
- **Score every finding by confidence.** A reviewer rates each finding from 0 to 100 and reports only those at a confidence of 70 or higher. The one exception is a Critical finding, which may be reported at 50 or higher, so a serious but uncertain risk is still surfaced.
- **Zero findings is a valid result.** After a thorough review, finding nothing is an acceptable outcome. A reviewer does not inflate nits, invent issues, or expand its scope to avoid an empty report.

## The external reviewers

Alongside the built-in lanes, cc-review can call external command-line review tools and parse their output into the same finding shape, so a tool's results sit beside the agents' findings in one report. Four are bundled, and they differ in whether a fresh install runs them.

- **CodeRabbit** runs by default.
- **Codex CLI** runs by default.
- **Copilot** is off by default.
- **Greptile** is off by default. It also needs the repository connected to a Greptile organization, and it spends credits on every run, which is another reason it stays off until you turn it on.

An external reviewer runs only when its command-line tool is both installed and enabled, so the two that are on by default still contribute nothing on a machine where their tools are absent.

## The gator persona

Gator is an opt-in reviewer that takes the stance of a principal engineer rather than a single-lane specialist. Its reviewer id is `gator`, and it ships disabled, so a default run never includes it. It was ported from the NVIDIA OpenShell project.

Where the six lanes each hold one concern, Gator balances three at once.

- **Pragmatism**: is the solution as simple as the problem allows, or is it over-engineered?
- **User empathy**: how the change lands on the people who use, operate, and maintain the system.
- **Security**: threat surfaces, trust boundaries, and input validation, weighed against established frameworks.

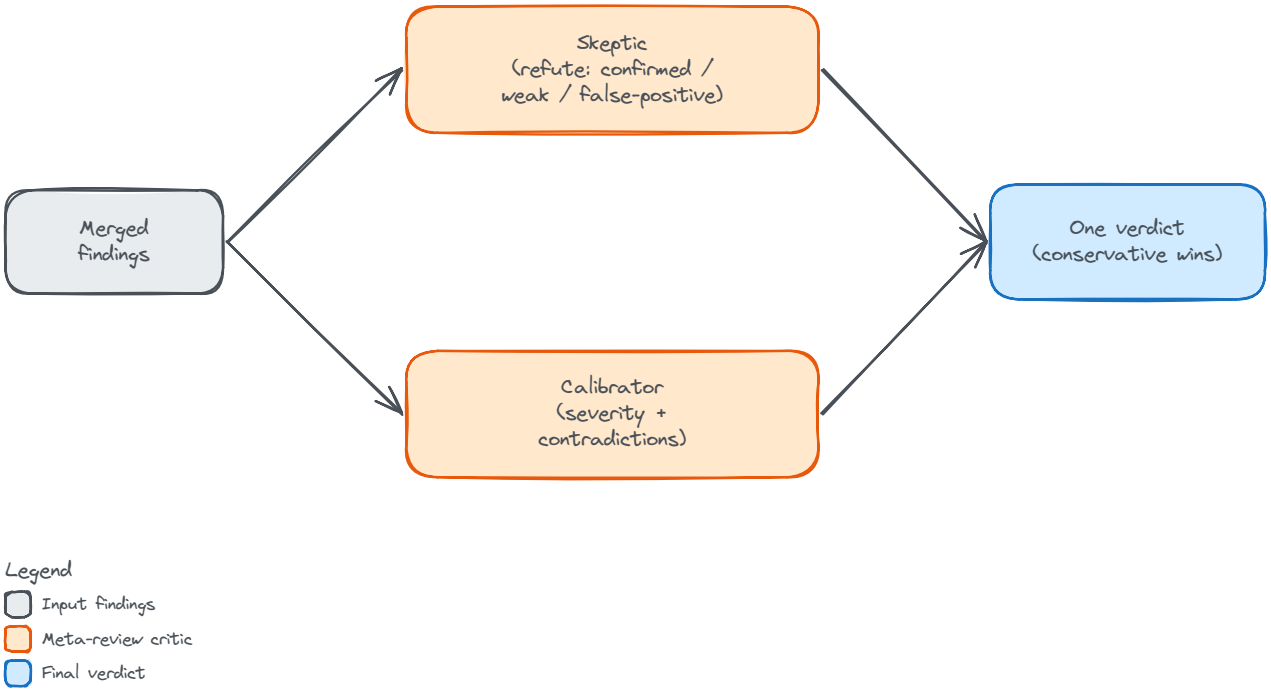
What sets Gator apart is its evidence contract. A blocking finding, one it rates Critical or Warning, is only allowed to block when its failure scenario carries a full chain of evidence. That chain names the invariant the change violates, the prerequisite capability to reach it, the entry point and the sink, the behavior before and after the change, the observable impact, why this change owns the problem, a concrete requested fix, and a minimal deterministic reproducer. A candidate that cannot meet every item is not dropped; it is reported as a non-blocking Suggestion that names the evidence it is missing.

The contract is what lets a single broad reviewer stay as disciplined as the focused lanes: an opinion only earns the power to block when it can prove itself.

Once a run collects every reviewer's findings, two meta-review critics judge them before anything is acted on. The meta-review page covers that stage.

# Meta-review

Once the panel's findings are merged into one list, two critics read that list before cc-review acts on any of it. The first decides whether each finding is real. The second decides whether its severity is right and whether the reviewers contradict one another. Both run at the same time over the same input, and cc-review combines their separate judgments into one outcome for every finding.




## One de-duplicated list

The critics do not judge every raw finding. Before they run, cc-review merges the findings from all reviewers and removes the duplicates. When two findings sit on the same file, cover overlapping lines, and share a category, cc-review folds them together. It keeps the one with the longer description, records the other reviewer in the finding's `also_reported_by` field, and takes the higher of the two severities and the higher confidence. A reviewer marked as isolated is held out of this cross-category step, so a specialized lane keeps its own count. That clean list is what both critics evaluate.

## The Skeptic

The Skeptic asks one question of every finding: is it real? It reads the code each finding points at and tries to refute it, looking for a guard the original reviewer missed, documented behavior that makes the finding moot, or a plain misreading of the code. Its stance is adversarial by design. It assumes a finding is wrong until the code proves it right.

Each finding comes back with one of three verdicts. It is **confirmed** when the code clearly shows the issue and nothing refutes it, and **weak** when the problem is plausible but the evidence does not support the claimed severity. It is a **false positive** when the code does not have the described issue at all. The bar for a false positive is deliberately high. The Skeptic may call a finding a false positive only when it can cite the specific code that disproves it, and it is told not to mark findings false just to shrink the list.

When it cannot evaluate a finding at all, it keeps the finding and marks it confirmed, erring on the side of caution. The effect of these rules is that the Skeptic removes few findings outright. Most of what it does is label, not delete.

The Skeptic does not touch severity, contradictions, or reviewer scores. Those belong to the Calibrator.

## The Calibrator

The Calibrator takes the same list and judges proportion rather than truth. For each finding it checks whether the claimed severity matches the evidence, against fixed definitions.

- **Critical** for data loss, a security breach, or a crash or silent corruption on a normally reachable path.
- **Important** for incorrect behavior or reliability problems that need specific but plausible conditions.
- **Minor** for a code-quality concern unlikely to reach a user.
- **Notable** for an observation about design rather than a defect.

When a severity is out of proportion, the Calibrator records a corrected level together with the code evidence behind it.

It then scans across reviewers for contradictions: two findings on the same stretch of code, from different reviewers, that make opposing claims. One says a value can be null while another says it is always set; one flags a race condition while another asserts thread safety. Two findings about different aspects of the same code, one about naming and one about logic, are not a contradiction.

Finally, the Calibrator scores each reviewer on the quality of its findings, estimating a precision, a signal-to-noise ratio, and the count it produced. Those scores are what let the report tell you which lanes were pulling their weight on this change.

## When the two disagree, the stricter call wins

cc-review dispatches the Skeptic and the Calibrator as two subagents in the same step, so they run in parallel and reach their judgments independently. Because the two look at different things, those judgments can diverge, and cc-review combines them in a fixed order. It first drops every finding the Skeptic called a false positive, then records the verdict on each finding that remains, then applies the Calibrator's corrected severities.

Where the two conflict, the more conservative outcome wins. When the Skeptic confirms a finding but the Calibrator downgrades its severity, the downgrade stands. A finding the Skeptic marked weak drops as well, to the Calibrator's level or, when there is none, by a single step with Minor as the floor. cc-review annotates contradictions on the findings rather than resolving them, so a reader can see where the reviewers disagreed.

If either critic fails or times out, cc-review skips the whole stage and lets every finding through unchanged rather than dropping the run.

## Why it waits for enough findings

Meta-review is gated. It runs only when it is enabled, which it is by default, and when the merged list reaches a minimum count, five findings by default. A smaller run skips the critics and goes straight to the gate. The threshold matches what the critics are for. Refuting, re-scoring, and cross-checking a handful of findings costs two extra subagents and buys little, while the same work over a long, noisy list is where it earns its place. The review pipeline page shows where this stage sits in a full run.

# Gate and fix

By the time the findings reach this stage, meta-review has filtered them into one list (see Meta-review). cc-review now has to turn that list into a decision: is the change ready, or does something still need fixing first? Two mechanisms do that. The gate reads the list and returns a verdict, and the fix loop tries to clear the findings that fail the gate so the verdict can flip to a pass.

## The gate

The gate is a count, not a judgment. cc-review counts the findings whose severity is one that blocks a merge. Which severities those are comes from a configuration setting, `REQUEST_CHANGES_SEVERITIES`, which holds Critical and Important by default. If the count is zero, the review is a **gate pass**. If the count is above zero, the review is a **gate fail**, and it stays failed unless the fix loop clears those findings within its round limit.

Minor and Notable findings fall outside that set, so neither one blocks a pass on its own. Notable findings are informational by definition, and Minor findings are reported for the record, yet the gate does not weigh either. A change can therefore carry a long tail of small suggestions and still pass, as long as nothing Critical or Important remains.

On a follow-up review the gate also accounts for concerns carried over from earlier rounds. A material concern that an earlier review raised and that is still open keeps the gate from a clean pass, even when the new changes add no findings of their own. An unresolved question from a previous round counts as unfinished work rather than as something the new diff quietly settled.

## The fix loop

When the gate fails and the run is set to fix, cc-review enters an autonomous fix loop. The loop runs a bounded number of rounds. The limit comes from the `max_fix_rounds` configuration setting and defaults to 3.

Each round has the same shape. cc-review gathers the findings it will act on and groups them by file. The Critical and Important findings are always in scope; when the run asked for all fixes rather than only the blocking ones, every Minor and Notable finding that carries a concrete suggested fix joins them. Inside each file, cc-review applies the fixes from the bottom of the file upward, working through the findings in descending line order. Editing the lower lines first would shift the line numbers of the findings still waiting above them, so starting from the bottom keeps every remaining line reference accurate.

Once the edits are staged, cc-review runs the project's test suite if it could detect one. A passing suite lets the round proceed. A failing test does not: cc-review turns each failure into a new Critical finding with the category `regression`. A fix that breaks the build cannot pass itself off as a clean result, because it returns as a fresh blocking finding in the same loop.

cc-review then re-reviews only the files the round changed. It re-dispatches the same reviewers that ran the first time, restricted to the modified files, and reuses the earlier decisions about which reviewers were enabled rather than running discovery again. The new findings are merged with the Minor

findings still on the list, and the gate runs once more. If Critical and Important are both at zero, the loop passes and stops early.

If the blocking findings are not cleared, the loop tries again, up to the three-round default. When it reaches that limit with findings still open, the gate-fail result stands and the review reports what remains instead of looping on. The loop exists to clear the issues it can fix automatically, not to force a pass.

The loop applies its edits to the tree under review, but whether those edits reach a pull request depends on push permission. cc-review commits the fixes in the PR worktree and pushes them to the PR branch, but that push can be refused when you lack access, the branch moved since the review started, or it fails for another reason. When that happens, cc-review keeps the fix worktree so the commit is not lost and does not post a review for code that is not on the PR.

## Follow-up reviews converge

A pull request is rarely reviewed only once. When you ask cc-review to review a PR again after pushing more commits, it does not start over. A follow-up reviews the changes since your last trustworthy completed run, including the code you changed to address earlier findings.

Earlier concerns are neither forgotten nor silently cleared. cc-review verifies each one explicitly against the current code and keeps it in its original thread until it is addressed. A concern that does not reappear in the new round never counts as proof that it was fixed. cc-review also remembers the earlier discussion, both its own comments and your replies, so it continues an existing concern instead of opening a duplicate, and it explains itself when it reverses advice it gave before.

The aim of a follow-up is to converge, not to accumulate. New findings focus on material problems, so optional polish does not open another round, and a critical finding on code the latest commits did not touch needs concrete evidence before it is raised. Once the verification is done and nothing substantive is left, the report can conclude that there are **no outstanding material concerns**. That is a legitimate result, not a sign of an incomplete review. A clean conclusion still does not approve or merge the PR on its own. It tells you the change is clear, and the decision stays with you.

To see where the gate and the fix loop sit in a full run, from discovery through the final report, see The review pipeline.

# Measured results

The pages before this one describe how cc-review is built and what each stage is meant to do. This one reports how well it has done it in practice. Everything below is a measurement taken at a point in time, not a guarantee of future behavior, and every figure carries the date it was measured. The numbers come from two recorded data sets: the labeled findings from cc-review runs on the public NVIDIA/OpenShell pull requests (`openshell-findings-labeled.jsonl`), and an aggregate of the maintainer's own local review reports (`local-reports-aggregate.json`). Read them as evidence from those runs, not as a specification of what cc-review will do on your change.

## Findings on the public OpenShell pull requests

The clearest public evidence is a set of cc-review runs on the NVIDIA/OpenShell repository, recorded in the labeled OpenShell findings data set. As measured in early November 2026, those runs produced 208 inline findings across 20 public pull requests, and the data set labels each finding with what became of it.

The two fix outcomes dominate. 77 findings were fixed with the fix verified in the code, and another 40 were fixed as reported by the author, so 117 of the 208 findings, about 56 percent, were addressed by a fix. The rest break down as 28 resolved without an explicit reply, 27 still open, 16 deferred, 16 that went stale with no reply, 3 withdrawn, and 1 rejected. These counts are the state of those threads as measured in early November 2026, attributed to the same OpenShell findings data set; the open and deferred ones in particular can still change as the pull requests move.

Because the OpenShell repository is public, you can read the threads yourself. cc-review's findings appear on pull request #5082, pull request #5083, and pull request #4789, among the 20 covered by the measurement.

## Fix rate across the maintainer's local runs

The second data set is an aggregate of local cc-review reports kept by the maintainer. As measured in early November 2026 it covers 147 recorded reports, the dated ones running from late March to early November 2026. For each report and severity it records how many findings were found and how many were fixed.

The headline measurement is the blocking severities. Across those 147 reports, and attributed to the local reports aggregate, cc-review found 632 Critical and Important findings and fixed 585 of them, about 93 percent, as measured in early November 2026. These are the severities the gate blocks on (see Gate and fix), so this is the rate at which the fix loop cleared the issues that would otherwise hold a change back. Counting every severity together, the same reports found 1901 findings and fixed 979, about 51 percent. The gap between the two rates is expected: Minor and Notable findings fall outside the gate, so many are reported for the record and left for you to judge rather than fixed automatically.

## What meta-review did in the measured reports

A smaller subset of the local reports ran with meta-review recorded in full. As measured in early November 2026 that subset is 7 reports covering 132 findings, again drawn from the local reports aggregate. In those reports the Calibrator recorded 16 severity adjustments, for example downgrading an Important finding to Minor, while the Skeptic removed none of the 132 findings outright as false positives.

That last number is the honest limit of the current meta-review, not a selling point. The Skeptic's role, by its own instructions, is to read the code behind each finding and judge it confirmed, weakly supported, or a false positive. In the measured subset it confirmed or weakened findings but struck none of them, so the only severity changes meta-review made in those reports came from the Calibrator's 16 downgrades (see Meta-review). Read the measured removal count of zero as a reminder that the Skeptic is cautious by design: in these runs no finding was a clear enough false positive for it to drop.

## How to read these numbers

Every figure on this page is a dated measurement from the two recorded data sets, taken in early November 2026. They describe cc-review's behavior on the OpenShell pull requests and on the maintainer's own changes up to that date, and they will drift as the tool, the reviewers, and the code under review change. Treat them as evidence that cc-review clears most of the blocking issues it raises and that its reviews have led to real fixes in public code, not as a promise of a particular fix rate on your next review.

# Extending cc-review

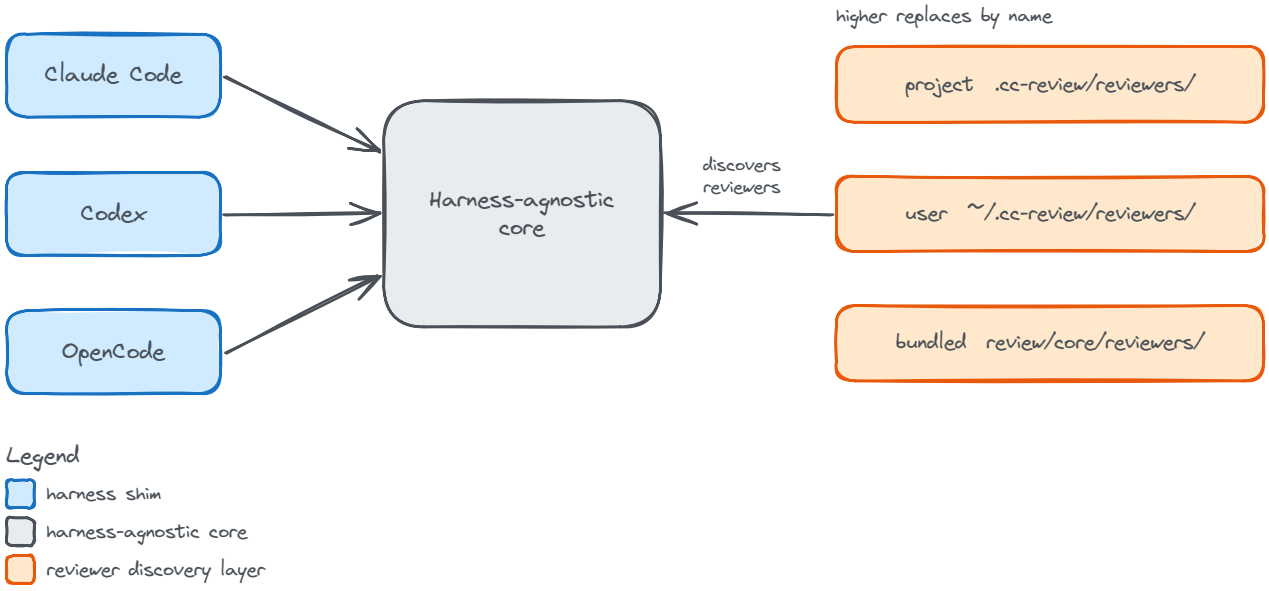
# Extending cc-review

cc-review ships with a panel of reviewers, and the panel is open. This section is for the Plugin author who wants to add to it: a reviewer that reads your changed files and reports findings in an area you care about, a wrapper around an external review tool so its output lands in the same merged report, or a change to which reviewers run and which ones you trust. Each of these is a small, local addition, because the core discovers reviewers on every run rather than holding a fixed list.

## How the pieces fit

cc-review keeps the review logic in one place and wires it into each harness separately. The `review/` directory is a single distributable that holds a harness-agnostic core alongside a native manifest for each of the three harnesses it supports: Claude Code, Codex, and OpenCode. Every harness reads its own manifest and reaches the same core. A thin command shim resolves the core directory at run time and delegates to the shared command files, and it adds no review logic of its own, so what you build to extend cc-review lives in the core, not in any one harness.

A reviewer is the unit you add, and the core carries no built-in list of them. On every run, `discover-reviewers.sh` scans three directories for reviewers and emits the set to dispatch. It reads the project layer `.cc-review/reviewers/` in the checkout under review, the user layer `~/.cc-review/reviewers/` in your home directory, and the bundled layer that ships inside the core. The layers have a fixed precedence, project over user over bundled, and a higher layer replaces a lower one of the same name. That is how a reviewer you write takes its place next to the built-in panel, and how a project or user reviewer can stand in for a bundled one by reusing its name.




The pages, in reading order:

- Write a prompt reviewer builds the simplest reviewer, one Markdown file with a short frontmatter block and a prompt body, and runs it alongside the built-ins.
- Write a command reviewer wraps an external command-line tool so its output becomes cc-review findings, walking through the bundled Greptile reviewer field by field.
- Trust and configuration covers the trust model that stops a change from steering its own review, and how you turn reviewers on and off in config and for a single run.

# Write a prompt reviewer

A reviewer is a plugin, and a prompt reviewer is the simplest kind: one Markdown file with a short YAML frontmatter block and a prompt body. You add one by dropping that file into a directory the review scans, and no core file, configuration template, or command definition changes. The body is the instruction set a fresh-context review agent follows when it reads your changed files and reports findings.

Discovery runs on every review and scans three layers: your project's `.cc-review/reviewers/`, your personal `~/.cc-review/reviewers/`, and the reviewers bundled with the plugin. A project reviewer lives with the code it reviews. A reviewer in `~/.cc-review/reviewers/` follows you across every project. When the same name appears in more than one layer, the higher layer wins, so a project or user reviewer can replace a bundled one by reusing its name.

The steps below build an accessibility reviewer for UI code.

1. **Create the file.** For a reviewer that belongs to one project, create the reviewer directory in the repository root and add a file named for the reviewer:

   ```
   mkdir -p .cc-review/reviewers
   ```

   Then create `.cc-review/reviewers/accessibility.md`. For a reviewer you want in every project, create it under `~/.cc-review/reviewers/` instead.

2. **Write the frontmatter.** A prompt reviewer needs two fields and takes two more that are worth setting:

   - `name` (required): the reviewer's identifier. It must be lowercase letters, digits, and hyphens, and it must equal the file basename, so `accessibility.md` carries `name: accessibility`. A short list of names is reserved because they collide with command flags, among them `only`, `fix`, `pr`, and `spec`.
   - `kind` (required): `prompt` for a reviewer whose body is the prompt the agent runs.
   - `display_name` (optional, defaults to `name`): the label shown in the console summary and the findings report.
   - `category` (optional, defaults to `name`): written to the `category` field of every finding this reviewer reports. It too is lowercase letters, digits, and hyphens.

   Unknown fields are ignored rather than rejected, so the format can grow without breaking older reviewers. The full contract is `review/core/schemas/reviewer.schema.json`.

3. **Write the prompt body.** The body is the reviewer prompt. Write it for an agent that reads the changed files itself and reports findings, and do not mention harness tools. Give it three parts: a role (what this reviewer is responsible for), a scope (what it is not responsible for, so it stays in its lane), and a checklist of concrete things to look for. Report severities directly as Critical, Important, Minor, or Notable.

> **Note**
>
> At dispatch the core prepends a shared preamble (anti-sycophancy, distrust, confidence scoring, and the output format), and it is always applied with no opt-out. Do not repeat those rules in your body, and remove anything that conflicts with them, such as instructions to praise good code. Your body follows the preamble, and after it come the review target, the spec and review hints when present, and the output contract.

The complete `.cc-review/reviewers/accessibility.md` looks like this:

```
---
name: accessibility
display_name: "Accessibility"
kind: prompt
category: accessibility
---
You are the ACCESSIBILITY REVIEW AGENT.

YOUR ROLE: You ARE responsible for finding accessibility defects in UI code:
missing alternative text, unlabeled form controls, keyboard traps, color-only
signaling, and ARIA misuse.

YOU ARE NOT RESPONSIBLE FOR: correctness bugs, security, performance, or
test quality. Stay in your lane.

For each changed file that renders UI, check:
- Every image and icon has meaningful alternative text or is marked decorative
- Every form control has an associated label
- Interactive elements are reachable and operable with the keyboard
- Information is not conveyed by color alone
- ARIA roles and attributes match the element's behavior

Report each defect as a finding with the exact element and line.
```

4. **Commit the file and run a review.** Discovery picks the reviewer up on the next run, dispatches it alongside the built-ins with the shared preamble prepended, and shows its row in the console summary (here "Accessibility"). Its findings carry `source_agent: accessibility` and `category: accessibility`. Run only your reviewer with `/cc-review:review --only accessibility`, or turn it off for one run with `/cc-review:review --no-accessibility`.

Check: the file sits in the project reviewer directory that discovery scans, so the next review runs it.

```
ls .cc-review/reviewers/accessibility.md
```

Project prompt reviewers run without any trust step. A reviewer that wraps an external command-line tool uses `kind: command` instead of `kind: prompt`.

> **Note**
>
> This page was verified against cc-review 1.4.0.

# Write a command reviewer

A command reviewer wraps an external command-line tool, such as a linter or a hosted review service, so its output becomes cc-review findings that join the ones the built-in agents report. It uses `kind: command` rather than `kind: prompt`. In place of a prompt, its frontmatter declares how to detect the tool, how to run it, and how to turn its output into findings, and the body holds parsing notes for the orchestrator.

cc-review ships Greptile as a bundled command reviewer in `review/core/reviewers/greptile/`. The reviewer wraps the `greptile` CLI to review the committed diff and fold the results into the report. Its directory is a complete worked example, so the steps below walk through it field by field. Build your own command reviewer the same way: drop its directory into `.cc-review/reviewers/` for one project, or `~/.cc-review/reviewers/` for every project.

1. **Use the directory form.** A command reviewer that ships a parser script uses the directory form, a `reviewer.md` beside the parser in a directory named for the reviewer. The Greptile reviewer's directory holds `reviewer.md` and a parser named `parse.sh`.

2. **Declare the frontmatter.** The frontmatter is the whole contract for running the tool. Greptile's reads:

   ```
   ---
   name: greptile
   display_name: "Greptile"
   kind: command
   category: external
   enabled: false
   detect: "command -v greptile"
   auth_check: "greptile whoami >/dev/null && greptile init --json >/dev/null"
   invoke: "greptile review --json --agent -b {base_branch}"
   timeout_seconds: 540
   severity_map:
     P0: Critical
     P1: Important
     P2: Minor
     P3: Notable
   output:
     format: parser
     parser: parse.sh
   ---
   ```

   Each field has a job:

   - `kind: command` marks the reviewer as a tool wrapper. `name`, `display_name`, and `category` behave as they do for a prompt reviewer, and here `category: external` is written to every Greptile finding.
   - `enabled: false` leaves the reviewer off until you turn it on, because every Greptile run spends credits (a base review costs one).
   - `detect` is a shell check that runs before dispatch. Exit 0 means the tool is available; any other exit skips the reviewer with the reason `not available`. Greptile's `command -v`

`greptile` confirms the CLI is on `PATH`.

- `auth_check` is an optional shell check that runs after `detect`. A non-zero exit skips the reviewer with `not authenticated`, and the last line the check wrote to stderr is appended to the reason. Greptile's check confirms a login and that the repository is connected to a Greptile organization; `greptile init --json` exits 1 for a repository that is not connected, so an unconnected repository is skipped rather than failed.
- `invoke` is the command template. cc-review substitutes three placeholders, `{base_branch}`, `{files}`, and `{pr_number}`. Greptile uses only `{base_branch}`; `--json` selects machine output and `--agent` suppresses prompts, browser sign-in, and update notices.
- `timeout_seconds` is the wall-clock bound on the invoke command (the default is 120). Greptile sets 540, because a base review of a roughly 3,000 line diff took about 200 seconds and the harness caps a single call at 600 seconds.
- `severity_map` translates the tool's native severity names into cc-review's four levels: Critical, Important, Minor, or Notable. Greptile maps `P0` through `P3`. A native value absent from the map passes through only when it is already a cc-review severity, and is otherwise dropped with a warning.
- `output.format` says how raw output becomes findings, one of `json`, `delimited`, or `parser`. Greptile prints a single JSON object that needs reshaping, so it uses `parser` and names `parse.sh`.

3. **Write the parser.** `greptile review --json` prints one JSON object with a `comments` array, alongside summary fields that cc-review ignores. A `parser` script reads that raw output on standard input and prints cc-review's `json` finding shape on standard output. The script runs through its own shebang, so any language works. The core of `parse.sh` builds each finding from one comment inside a `jq` `map(...)`:

```
| {
    file: .path,
    line: $line,
    line_end: $line_end,
    severity: .severity,
    description: ($first | gsub("^\\s*\\*\\*|\\*\\*\\s*$"; "")),
    rationale: $body,
    confidence: 75
  }
+ (if (.suggestion | type) == "string" and .suggestion != "" then {fix: .suggestion} else {}
end)
```

The parser builds each finding like this:

- `path` becomes `file`, and `startLine` and `endLine` become `line` and `line_end`.
- The comment body's bold title line becomes `description`, and the whole body becomes `rationale`.
- A non-empty `suggestion` becomes `fix`.
- `confidence` is fixed at 75 for every finding, because Greptile's top-level confidence is a merge-readiness score for the whole change, not a per-finding value.

When standard input is empty, holds more than one value, or is not a review object with a `comments` array, the parser exits non-zero, and the review records a parse error rather than

reading a failed run as a clean one.

4. **Enable it and check the preconditions.** Greptile is off by default, so a plain run skips it. A `kind: command` reviewer in a project's `.cc-review/reviewers/` also runs none of its commands until you trust it. Enabling a reviewer and trusting a project command reviewer are both covered in Trust and configuration. Greptile reviews committed changes only, and the repository's remote must be connected to a Greptile organization, or the `auth_check` skips it.

Check: the parser turns one Greptile comment into a cc-review finding that carries the file path.

```
printf '%s' '{"comments":
[{"path":"src/app.js","startLine":10,"endLine":12,"severity":"P1","body":"**Null deref**\nvalue may
be null","suggestion":"guard the value"}]}' | review/core/reviewers/greptile/parse.sh
```

> **Note**
>
> This page was verified against cc-review 1.4.0.

# Trust and configuration

The project reviewer layer is read from the checkout being reviewed, so whoever authors a change also controls those files. Discovery applies a trust model so a change cannot steer or exploit its own review. Once you understand that model, the second half of this page covers how you turn reviewers on and off, in config and per run. For how reviewers themselves are written, see Write a prompt reviewer and Write a command reviewer.

## The trust model

Three guards protect a review from the change under review.

**A reviewer the change edits does not judge it.** When a project reviewer's file is part of the change under review, discovery ignores that reviewer for this run. The guard covers the reviewer's `.md`, any file of its plugin directory, a reference it declares, and its parser. The bundled or user reviewer of the same name runs instead, and a reviewer the change adds is listed as `skipped (changed in the reviewed change)`. A new or edited project reviewer takes effect for every later change, once it is committed and is no longer part of a diff.

**Project command reviewers need your trust.** A `kind: command` reviewer in `.cc-review/reviewers/` runs shell commands (`detect`, `auth_check`, `invoke`, and its parser) with your credentials. cc-review runs none of them until you trust that reviewer by its sha256 hash in `~/.cc-review/trusted-reviewers`. Until then it shows as `skipped (untrusted project command reviewer (trust with: <name> <sha256>))`. The hash covers the reviewer file, or for a directory-form reviewer every file in its directory (its parser and any helper the parser sources), so any edit to those files needs a new trust line. A reviewer directory that holds a symlink is never trusted. The trust file is read only from your home directory, and a trust file placed inside the project under review is ignored with a warning, since the change could otherwise trust its own reviewers. Project prompt reviewers, and every reviewer in the user and bundled layers, need no trust.

**Command reviewers skip an untrusted fork checkout.** When the tree under review is a cross-repository (fork) pull request's code, cc-review marks it as an untrusted checkout. This happens for an isolated PR checkout and for a bulk worker reviewing a fork PR. Because an external tool run inside that checkout could load the PR's own agent instruction files (`AGENTS.md` and the tool's own instruction files in the repository), command reviewers do not run there by default. Each one that is still ready becomes `skipped (untrusted checkout)`. Prompt reviewers still run, and neither `--only` nor `--external` overrides the skip. Opt a single command reviewer back in with `allow_untrusted_checkout.<name>: true` in config.

## Trust a project command reviewer

A project command reviewer is skipped until you vouch for the exact version in the checkout. You do that by adding one line, the reviewer's name and its current sha256 hash, to your home trust file. cc-review generates that line for you so you never compute the hash by hand.

1. **List the trust line.** Run `/cc-review:init` (or any review). Discovery reports each untrusted project command reviewer together with the line that trusts its current version:

   ```
   Untrusted project command reviewers run nothing until you trust them. To trust the current
   version, add these lines to ~/.cc-review/trusted-reviewers:
   lint-service 3f1c...e9a0
   ```

2. **Add the line to your trust file.** Copy each `<name> <sha256>` line into `~/.cc-review/trusted-reviewers`, one per line. Blank lines and lines beginning with `#` are ignored, so you can annotate the file:

   ```
   # ~/.cc-review/trusted-reviewers: <name> <sha256>, one per line
   lint-service 3f1c...e9a0
   ```

3. **Re-run the review.** Discovery now matches the hash, so the reviewer's `detect` and `auth_check` probes run and it moves from untrusted to ready (or to a detection-based status such as `not available`). Any later edit to the reviewer or its parser changes the hash, so the reviewer returns to untrusted until you add the new line.

> **Caution**
>
> Trusting a command reviewer lets the project run commands with your credentials. `/cc-review:init` prints the trust lines but never adds them for you. Add a line only after you have inspected the reviewer and everything it runs. Keep all of that inside the reviewer's own directory, because a script `invoke` or the parser calls from elsewhere in the repository is not covered by the hash.

Check: the trust file holds a reviewer name and a 64-character sha256 digest on one line.

```
grep -Eq '^[a-z0-9-]+ [0-9a-f]{64}$' ~/.cc-review/trusted-reviewers && echo trusted
```

# Enable or disable reviewers

Every discovered reviewer runs by default except the ones whose own frontmatter ships disabled (among the bundled set, copilot, gator, and greptile). Turn any reviewer on or off for a whole project in config, or override the set for a single run with flags.

1. **Set a config key.** In `.cc-review/config.yml` (project) or `~/.cc-review/config.yml` (user), give each reviewer you want to change a `reviewers.<name>` key:

   ```
   reviewers:
     security: false     # turn a bundled reviewer off
     gator: true         # turn an opt-in reviewer on
   ```

   A reviewer without a key keeps the `enabled` default from its own frontmatter.

2. **Override for one run with flags.** Flags beat config and apply after discovery:

- `/cc-review:review --only correctness,security` runs only the named reviewers. `--only` re-enables a config-disabled reviewer, but never a reviewer that discovery skipped or marked invalid.
- `/cc-review:review --no-security` subtracts one reviewer from the run. `--no-<name>` for an invalid reviewer prints a warning and the review continues.
- `/cc-review:review --no-external` drops every external command reviewer at once.

The resolution order settles conflicts: CLI flags win over `--config`, then `--profile`, then project config, then user config, then the built-in defaults, then each reviewer's own `enabled` default.

Check: the config template you copy documents the `reviewers` key.

```
grep 'Enable or disable any discovered reviewer' review/config/config-template.yml
```

> **Note**
>
> This page was verified against cc-review 1.4.0.

# Reference

# Reference

The reference section is where you look up an exact value rather than follow a workflow. It gathers the flag, profile, configuration, and finding-schema details that the task and concept pages mention only in passing. The Using cc-review and How it works sections explain when and why to reach for these; this section gives the exact values.

The pages:

- CLI reference lists every flag for the four cc-review commands and what each bundled profile sets.
- Configuration and finding schema lists the configuration keys, the order its layers resolve in, and the fields of a finding.
- Invoking cc-review across harnesses shows how to call a command on Claude Code, Codex, and OpenCode.

# CLI reference

Every flag for the four cc-review commands, and what each bundled profile sets. The tables reproduce the flag descriptions from the project README at the pinned version. For how to use these commands, see Review your changes, Triage pull request comments, and Inbox and issue assessment.

## review

Flags for the `/cc-review:review` command.

| Flag | Description |
|---|---|
| `--pr <number>` | Review a specific PR's diff |
| `--full` | Force full PR scope, including already-reviewed bulk candidates; retains draft/filter exclusions |
| `--spec <path>` | Specification file for compliance checking |
| `--hints <path>` | Review hints file (default: `.cc-review/review-hints.md`) |
| `--output <path>` | Output report path (default: `./review-findings.md`) |
| `--config <path>` | Explicit config file |
| `--profile <name>` | Profile preset (`quick`, `thorough`, `ci`) |
| `--no-fix` | Skip autonomous fix loop |
| `--max-rounds <n>` | Maximum fix loop rounds (default: 3) |
| `--no-external` / `--external` | Disable or enable all command (external tool) reviewers |
| `--no-<reviewer>` | Skip one reviewer by name (for example `--no-codex`, `--no-test-quality`) |
| `--only <reviewer,...>` | Dispatch only the listed reviewers (overrides config, not preconditions) |
| `--no-meta-review` | Skip the skeptic/calibrator meta-review pass |
| `--sequential` | Run reviewers one at a time (default: parallel) |
| `--parallel` | Run reviewers in parallel (the default; accepted for compatibility) |
| `--bulk` | Review several PRs in one run |
| `--all-open` | Bulk: select every open PR |
| `--search <query>` | Bulk: select the open PRs matching a GitHub search |
| `--limit <n>` | Bulk: at most this many PRs (default: 10) |
| `--unattended` | Bulk: no questions, apply the fixed policy |

| Flag | Description |
| --- | --- |
| `--resume [<run-id>]` | Bulk: continue an unfinished run |

> **Note**
>
> With `--pr` on a branch other than the PR's own, the PR is checked out into a separate worktree under `.cc-review/pr-worktrees/` and reviewed there, so your working tree stays as it is.

# triage

Flags for the `/cc-review:triage` command.

| Flag | Description |
| --- | --- |
| `--pr <number>` | Target PR number |
| `--spec <path>` | Specification file for context |
| `--no-coverage-fix` | Skip automatic coverage remediation |
| `--idea-inbox <path>` | File for capturing out-of-scope ideas |
| `--bulk`, `--search`, `--limit`, `--unattended`, `--resume` | Triage several of your PRs in one run |

# inbox

Options for the `inbox` command.

| Option | Behavior |
| --- | --- |
| `--repo owner/name` | Select a repository explicitly, including outside a checkout. Default follows the invoking `gh` repository context. |
| `--type pr,issue,discussion` | Select any combination; repeat to combine selections. Default: all three types. |
| `--limit N` | Select the most recently updated items per type, from 1 to 100. Default: 10 per type. |
| `--since YYYY-MM-DD` | Filter every selected type by its last update. |
| `--help` | Show collector options without contacting GitHub. |

By default, PRs and issues are open, including draft PRs, with no recency cutoff. Discussions cover the last 90 days of activity, including answered and closed conversations. An explicit `--since` replaces that window and applies to all selected types.

# issue

Arguments and options for the `issue` command. The command takes an issue target as a number or a full issue URL.

| Argument or option | Behavior |
|---|---|
| `<number>` | Issue number to assess (for example `issue 42`) |
| `<url>` | Full issue URL to assess |
| `--repo owner/name` | Assess the issue in an explicitly named repository |
| `--help` | Show the command options |

> **Note**
>
> Issue assessment works on GitHub.com only, requires an explicit issue target, and rejects pull requests.

# Profiles

Three bundled profiles preset which reviewers run and how strict the gate is. Select one with `--profile <name>`. The tables below show exactly what each profile file sets; a setting a profile does not list keeps the base configuration default.

## Reviewers each profile enables

| Reviewer | `quick` | `thorough` | `ci` |
|---|---|---|---|
| correctness | yes | yes | yes |
| architecture | no | yes | yes |
| security | yes | yes | yes |
| production | no | yes | yes |
| test-quality | no | yes | yes |
| goal-alignment | no | yes | yes |
| coderabbit | no | yes | yes |
| copilot | no | yes | yes |
| codex | no | yes | yes |
| greptile | no | yes | no |
| gator | no | no | no |

> **Note**
>
> The `quick` profile also leaves user and project reviewers at their own enabled default; use `--only` for a strict allowlist. The `ci` profile disables greptile because it spends Greptile credits on every run.

## Thresholds, output, and fix loop

| Setting | `quick` | `thorough` | `ci` |
|---|---|---|---|
| `severity.min_confidence` | 70 | 80 | 70 |
| `severity.request_changes_severities` | Critical | Critical, Important, Minor | Critical, Important |
| `severity.auto_fix` | true | true | true |
| `output.verbosity` | quiet | normal | verbose |
| `max_fix_rounds` | 1 | 5 | 3 |
| `pr_posting.enabled` | not set | not set | false |
| `interactive` | not set | not set | false |

> **Note**
>
> These flags and profiles were verified against cc-review 1.4.0.

# Configuration and finding schema

This page lists the cc-review configuration keys, the order in which configuration layers resolve into one effective configuration, and the fields of a finding. For the command flags that can override these settings, see the CLI reference.

## Configuration file

cc-review reads YAML configuration from a project file at `.cc-review/config.yml` (which takes precedence) or a user file at `~/.cc-review/config.yml`. Each is copied from `review/config/config-template.yml`, and that same template is the built-in defaults layer of the merge.

```
reviewers:                # one key per discovered reviewer; omit a key to use the reviewer's
default
  correctness: true
  architecture: true
  security: true
  production: true
  test-quality: true
  goal-alignment: true
  coderabbit: true        # command reviewer (CodeRabbit CLI)
  copilot: false          # command reviewer (Copilot CLI)
  codex: true             # command reviewer (Codex CLI)
  greptile: false         # command reviewer (Greptile CLI, spends credits)
  gator: false            # opt-in principal-engineer reviewer

test:
  command: ""             # Auto-detected from Makefile/go.mod/package.json/pyproject.toml
  timeout_seconds: 300

triage:
  bot_profiles:
    - login: "coderabbitai[bot]"
      self_resolves: true
      auto_resolve: false
    - login: "copilot[bot]"
      self_resolves: false
      auto_resolve: true
    - login: "devin-ai-integration[bot]"
      self_resolves: false
      auto_resolve: true
    - login: "greptile-apps[bot]"
      self_resolves: false
      auto_resolve: true
  codecov:
    patch_threshold: 80
    auto_remediate: true

max_fix_rounds: 3
output:
  dir: "."
```

### Top-level keys

| Key | Shape | Purpose |
|---|---|---|
| `reviewers` | map of booleans | One key per discovered reviewer. Omit a key to use that reviewer's own default. |
| `test` | map | The test command and timeout the fix loop runs. |
| `triage` | map | Bot handling and Codecov thresholds for triage. |
| `max_fix_rounds` | integer | Maximum fix loop rounds. |
| `output` | map | Where reports are written. |

### reviewers

Each key under `reviewers` enables (`true`) or disables (`false`) one discovered reviewer. The values below are those set in the config template.

| Reviewer key | Template value | Notes |
|---|---|---|
| `correctness` | `true` | |
| `architecture` | `true` | |
| `security` | `true` | |
| `production` | `true` | |
| `test-quality` | `true` | |
| `goal-alignment` | `true` | |
| `coderabbit` | `true` | Command reviewer (CodeRabbit CLI) |
| `copilot` | `false` | Command reviewer (Copilot CLI) |
| `codex` | `true` | Command reviewer (Codex CLI) |
| `greptile` | `false` | Command reviewer (Greptile CLI, spends credits) |
| `gator` | `false` | Opt-in principal-engineer reviewer |

### test

| Key | Template value | Notes |
|---|---|---|
| `test.command` | "" (empty) | Auto-detected from Makefile, go.mod, package.json, or pyproject.toml. |
| `test.timeout_seconds` | `300` | |

### triage

| Key | Template value | Notes |
|---|---|---|
| `triage.bot_profiles` | four entries | The template lists `coderabbitai[bot]` (`self_resolves: true`, `auto_resolve: false`), `copilot[bot]`, `devin-ai-integration[bot]`, and `greptile-apps[bot]` (each `self_resolves: false`, `auto_resolve: true`). |
| `triage.codecov.patch_threshold` | `80` | |
| `triage.codecov.auto_remediate` | `true` | |

### max_fix_rounds and output

| Key | Template value | Notes |
|---|---|---|
| `max_fix_rounds` | `3` | Maximum fix loop rounds. |
| `output.dir` | `"."` | Directory reports are written to. |

> **Note**
>
> The legacy keys `agents.<name>` and `external_tools.<name>` are deprecated. They keep working for the migrated reviewers (`agents.test_quality` maps to `reviewers.test-quality`) and print a one-line warning that points to the new key.

## Resolution order

cc-review merges several configuration layers into one effective configuration. A higher layer overrides a lower one key by key. The order, from highest priority to lowest:

| Priority | Layer | Where it comes from |
|---|---|---|
| 1 (highest) | CLI flags | Flags passed on the command line (see the CLI reference). |
| 2 | `--config <path>` | An explicit configuration file. |
| 3 | `--profile <name>` | A bundled profile (`quick`, `thorough`, `ci`). |
| 4 | Project config | `.cc-review/config.yml` in the git root. |
| 5 | User config | `~/.cc-review/config.yml`. |

| Priority | Layer | Where it comes from |
|---|---|---|
| 6 | Built-in defaults | `review/config/config-template.yml`. |
| 7 (lowest) | Reviewer default | The reviewer's own `enabled` value. |

> **Note**
>
> The five file layers (built-in defaults, user config, project config, profile, and `--config` file) are assembled in `resolve-config.sh` and deep-merged from lowest to highest priority, so the explicit `--config` file wins among the files.

# Finding schema

Every reviewer reports findings in a common structure, validated against `finding.schema.json`. The gate and the meta-review act on a finding's `severity`, `confidence`, and `resolution`.

## Severity

A finding carries one severity, ordered `Critical > Important > Minor > Notable`.

| Severity | Rank |
|---|---|
| `Critical` | highest |
| `Important` | |
| `Minor` | |
| `Notable` | lowest |

For how severity drives the pass or fix decision, see Gate and fix.

## Confidence

`confidence` is an integer from 0 to 100. The minimum to report a finding is 70, lowered to 50 for `Critical` findings.

## Resolution and round tracking

| Field | Values or type | Meaning |
|---|---|---|
| `resolution` | `pending`, `fixed`, `unresolved` | Current resolution status. |
| `round_found` | integer (minimum 1) | Fix loop round when the finding was discovered (1 is the initial review). |

### Meta-review fields

These fields are present only when the meta-review ran.

| Field | Values or type | Meaning |
|---|---|---|
| `verdict` | `confirmed`, `weak`, `false-positive` | The Skeptic's assessment of finding validity. |
| `verdict_reasoning` | string | The Skeptic's code-cited justification for non-confirmed verdicts. |
| `calibrated_severity` | `Critical`, `Important`, `Minor`, `Notable` | The Calibrator's adjusted severity (present only when changed). |
| `calibration_reasoning` | string | The Calibrator's justification for the severity adjustment. |

For how the two critics produce these, see Meta-review.

## severity_map translation at merge

A reviewer plugin declares its metadata in frontmatter validated against `reviewer.schema.json`. When a command reviewer wraps an external tool whose severity vocabulary differs from cc-review's, the reviewer translates it with a `severity_map`.

`severity_map` is an object: each key is a native severity string, and each value is one of `Critical`, `Important`, `Minor`, or `Notable`. A native value not listed in the map passes through unchanged when it is already a cc-review severity; otherwise the finding is dropped with a warning.

> **Note**
>
> These configuration keys and schema fields were verified against cc-review 1.4.0.

# Invoking cc-review across harnesses

cc-review ships its commands as skills, one skill per command (`review`, `triage`, `inbox`, `issue`, and `init`), and each harness has its own way to call them. The call syntax is the only thing that changes between harnesses; the command itself does the same work wherever you start it.

**Convention:** the examples throughout this guide use Claude Code syntax, for example `/cc-review:review`. On another harness, translate the call using the table below.

## Calling a command on each harness

| Harness | How you call a command | Example (review) |
|---|---|---|
| Claude Code | The plugin-prefixed slash command `/cc-review:<command>` always works. The bare `/<command>` works only when no other installed command shares that name. | `/cc-review:review` |
| Codex | Type `$` followed by the bare skill name, with no plugin namespace: `$review`, `$triage`, `$inbox`, `$issue`, `$init`. You can also run `/skills` to pick from a list, or just describe the task and let Codex match the skill. There is no `$cc-review:review` namespace form. | `$review` |
| OpenCode | The bare slash command, with no namespace: `/review`, `/triage`, `/inbox`, `/issue`. | `/review` |

> **Important**
>
> The OpenCode commands do not register out of the box at this version. As of OpenCode v2.1.4, OpenCode discovers commands only from `.opencode/command/` in the project or `~/.config/opencode/commands/` globally. The cc-review OpenCode installer copies the command files to `.opencode/plugins/cc-review/command/`, a path stock OpenCode does not scan, so `/review` and the other commands will not appear until those files are placed in `.opencode/command/`. Check the cc-review project for a fix, or move the files into `.opencode/command/` yourself.

## How the Claude prefix is decided

On Claude Code, a skill is addressable both by its bare name and by its plugin-qualified name. The bare `/review` resolves as long as no other installed skill or command claims the name `review`. When two commands share a name, the bare form is ambiguous, and the plugin prefix picks the one you mean:

`/cc-review:review`. Because the prefixed form always resolves, it is the safe choice to write down and share.

## Translating examples

The rest of this guide writes commands in Claude Code form, `/cc-review:<command>`. To follow along on another harness, swap the call: use `$<command>` on Codex (for example `$review`) and `/<command>` on OpenCode (for example `/review`). The flags and the behavior are the same in every case.